

Basic Matrix Operations and Engineering Applications

1. What is a Matrix?

A **matrix** is a rectangular arrangement of numbers organized into rows and columns.

For example,

$$A = \begin{bmatrix} 2 & 4 & 6 \\ 1 & 3 & 5 \end{bmatrix}$$

This matrix has **2 rows and 3 columns**, so its size is

$$2 \times 3$$

A matrix with m rows and n columns is called an $m \times n$ matrix. As we have rows and columns, a matrix is usually 2-dimensional (2-D), but it can grow to 3-D, 4-D,... n -D, which will be usually called **tensor** in mathematics, **array** in computer programming.

Rank	Common Name	Indices Needed	Example
0	Scalar	0	42
1	Vector	1	[1, 2, 3]
2	Matrix	2	[[1,2],[3,4]]
3	Tensor (order 3)	3	[[[1,2],[3,4]], [[5,6],[7,8]]]
n	Tensor (order n)	n	...

2. Basic Matrix Terminology**Element**

The element in row i , column j is written as a_{ij} . For

$$A = \begin{bmatrix} 2 & 4 \\ 7 & 9 \end{bmatrix}$$

we have:

$$a_{11} = 2, a_{12} = 4, a_{21} = 7, a_{22} = 9$$

Square Matrix

A matrix with the same number of rows and columns:

$$A = \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$$

is a 2×2 square matrix.

Identity Matrix

The identity matrix has 1s on the main diagonal and 0s elsewhere:

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

It behaves like the number 1 in matrix multiplication:

$$AI = IA = A$$

3. Matrix Addition and Subtraction

Two matrices can be added or subtracted **only if they have the same dimensions**. If

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

then

$$A + B = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Similarly,

$$A - B = \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix}$$

Engineering application

Matrix addition is useful when combining quantities from multiple sources. For example, in structural engineering, matrices can represent forces or displacements at different locations.

4. Scalar Multiplication

A matrix can be multiplied by a single number called a **scalar**.

For example,

$$3A = 3 \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 12 & 15 \end{bmatrix}$$

Every element is multiplied by the scalar.

Engineering application: Scalar multiplication can represent scaling:

- changing the magnitude of a force, scaling a signal, changing the gain of a control system, converting units, scaling a geometric model.

5. Matrix Multiplication

This is one of the **most important matrix operations in engineering**.

Suppose

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

and

$$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Then

$$AB = \begin{bmatrix} 1(5) + 2(7) & 1(6) + 2(8) \\ 3(5) + 4(7) & 3(6) + 4(8) \end{bmatrix}$$

so

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Important rule

For

$$A_{m \times n} B_{n \times q}$$

Matrix Summary

matrix multiplication is possible only when $n = p$

The resulting matrix has dimensions $m \times q$

Example: If the two matrices are 2×3 , and 3×4

is possible and produces a 2×4 matrix.

But

$$(3 \times 4)(2 \times 3)$$

is **not possible**.

So in matrix operations, commutative law doesn't stand, i.e., $AB \neq BA$.

6. Matrix Multiplication Is NOT Element-by-Element Multiplication

This is an important distinction, especially when using MATLAB.

Matrix multiplication AB

uses the **row-by-column rule**.

Element-by-element multiplication

$$A \cdot B$$

multiplies corresponding elements.

For example,

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Element-by-element multiplication gives

$$A \circ B = \begin{bmatrix} 1(5) & 2(6) \\ 3(7) & 4(8) \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$$

Whereas,

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

In **MATLAB**:

$A * B$ % Regular Matrix multiplication, as shown in section 5.

$A .* B$ % Element-by-element multiplication, as shown in section 6.

This distinction is extremely important in engineering calculations.

7. Transpose

The **transpose** changes rows into columns, and columns to rows.

If

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

then

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Matrix Summary

Therefore, a 2×3 matrix becomes a 3×2 matrix.

MATLAB

$$A'$$

gives the transpose for a real-valued matrix.

Engineering applications

Transpose is commonly used in: vector and coordinate transformations, structural analysis, robotics, signal processing, least-squares calculations, machine learning.

For example, many engineering optimization problems involve

$$A^T A$$

8. Determinant

The **determinant** is defined only for a square matrix (number of rows is equal to the number of columns).

For a 2×2 matrix,

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

the determinant is

$$\det(A) = ad - bc$$

For example,

$$A = \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$$

gives

$$\det(A) = 2(5) - 3(4) = -2$$

MATLAB function `det(A)` is to calculate the determinant of the matrix.

Why is the determinant important?

If $\det(A) \neq 0$, then A is **invertible**. If $\det(A) = 0$, then A is **singular** and does not have an ordinary inverse.

Engineering applications

Determinants are useful for: checking whether a system of equations has a unique solution, coordinate transformations, stability analysis, calculating area and volume scaling, and finite element analysis.

9. Matrix Inverse

The inverse of A is written as

$$A^{-1}$$

and satisfies

$$AA^{-1} = A^{-1}A = I,$$

provided that A is invertible.

For example,

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Matrix Summary

has inverse

$$A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

provided

$$ad - bc \neq 0$$

MATLAB uses function `inv(A)` to find A^{-1} .

10. Matrix Division and Solving Linear Equations

A major application of matrices is solving systems of simultaneous equations.

Suppose we have

$$\begin{aligned} 2x + 3y &= 8 \\ 4x + y &= 6 \end{aligned}$$

We can write this as

$$Ax = b$$

where

$$A = \begin{bmatrix} 2 & 3 \\ 4 & 1 \end{bmatrix}, x = \begin{bmatrix} x \\ y \end{bmatrix}, b = \begin{bmatrix} 8 \\ 6 \end{bmatrix}$$

Thus,

$$\begin{bmatrix} 2 & 3 \\ 4 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 8 \\ 6 \end{bmatrix}$$

In MATLAB:

`A = [2 3; 4 1];`

`b = [8; 6];`

When solving engineering equations, MATLAB usually recommends using:

$$x = A \backslash b;$$

instead of

$$x = \text{inv}(A) * b;$$

The first approach is generally more efficient and numerically reliable.

11. Example: Electrical Circuit Analysis

Consider a circuit with several unknown currents.

Kirchhoff's laws can produce equations such as

$$\begin{aligned} 10I_1 - 2I_2 &= 5 \\ -2I_1 + 8I_2 &= 10 \end{aligned}$$

These can be written as

$$\begin{bmatrix} 10 & -2 \\ -2 & 8 \end{bmatrix} \begin{bmatrix} I_1 \\ I_2 \end{bmatrix} = \begin{bmatrix} 5 \\ 10 \end{bmatrix}$$

or simply

$$AI = b$$

MATLAB can solve this directly:

`A = [10 -2; -2 8];`

`b = [5; 10];`

`I = A\b;`

This approach scales naturally from 2 equations to thousands or millions of equations.

12. Example: Structural Engineering

In structural analysis, a very common equation is

$$Kx = F$$

where:

- K = stiffness matrix
- x = displacement vector
- F = force/load vector

For example,

$$\begin{bmatrix} 200 & -100 \\ -100 & 150 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 10 \\ 20 \end{bmatrix}$$

Solving the matrix equation gives the unknown structural displacements.

13. Important MATLAB Matrix Commands

For engineering students, these are some of the most useful commands:

Operation	MATLAB	Example
Create matrix	<code>[]</code>	$A = [12; 34]$
Addition	<code>+</code>	$A + B$
Subtraction	<code>-</code>	$A - B$
Scalar multiplication	<code>*</code>	$3 * A$
Matrix multiplication	<code>*</code>	$A * B$
Element multiplication	<code>.*</code>	$A.*B$
Transpose	<code>'</code>	A'
Determinant	<code>det()</code>	$det(A)$
Inverse	<code>inv()</code>	$inv(A)$
Solve $Ax = b$	<code>\</code>	$A \backslash b$
Element division	<code>./</code>	$A./B$
Matrix power	<code>^</code>	A^2

Matrix Summary

Operation	MATLAB	Example
Element power	<code>.^</code>	<code>A.^2</code>
Number of rows/columns	<code>size()</code>	<code>size(A)</code>
Largest element	<code>max()</code>	<code>max(A)</code>
Smallest element	<code>min()</code>	<code>min(A)</code>
Diagonal	<code>diag()</code>	<code>diag(A)</code>
Identity matrix	<code>eye()</code>	<code>eye(3)</code>
Zero matrix	<code>zeros()</code>	<code>zeros(3,3)</code>
One matrix	<code>ones()</code>	<code>ones(3,3)</code>

14. Key Concepts to Remember

For an engineering student, the most important ideas are:

1. **Matrix addition/subtraction:** same-sized matrices only.
2. **Matrix multiplication:** inner dimensions must match.
3. *** vs .*:**
 - `*` → matrix multiplication
 - `.*` → element-by-element multiplication
4. **Transpose:** rows become columns.
5. **Determinant:** helps determine whether a square matrix is invertible.
6. **Inverse:** satisfies $AA^{-1} = I$.
7. $Ax = b$: one of the most important matrix equations in engineering.
8. **MATLAB `A\b`:** preferred way to solve $Ax = b$ in most numerical applications.
9. Matrices are fundamental to **circuits, structures, control systems, robotics, signals, computer graphics, and AI**.